

Program No. 28

Objective: Write a program in Java for throw and throws Exception

Code Implementation (Source Code):

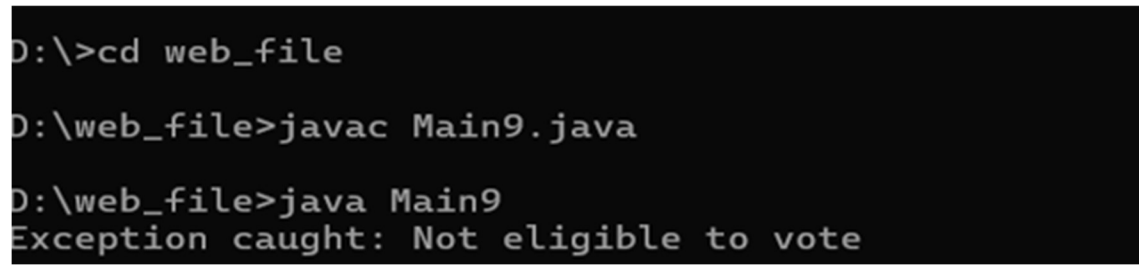
```
import java.util.*;

public class ThrowThrowsDemo {

    // Method using 'throws'
    static void checkAge(int age) throws Exception {
        if (age < 18) {
            // using 'throw'
            throw new Exception("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16); // change value to test
        } catch (Exception e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

Output :



```
D:\>cd web_file

D:\web_file>javac Main9.java

D:\web_file>java Main9
Exception caught: Not eligible to vote
```

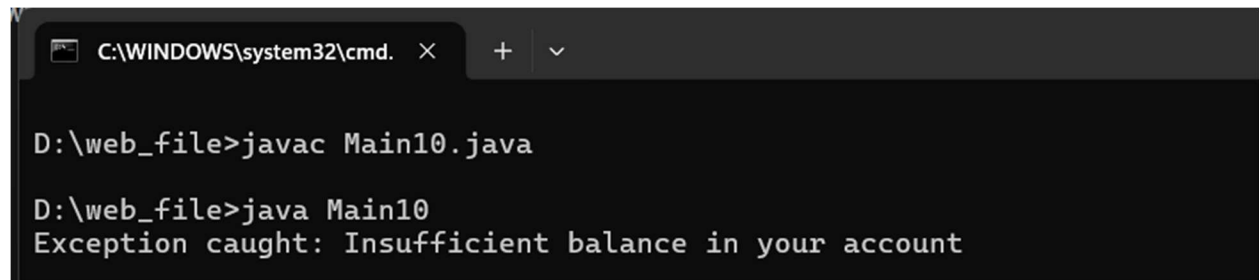
Program No. 29

Objective: Write a program in Java to throw your own Exceptions.

Code Implementation (Source Code):

```
class InsufficientBalanceException extends Exception {  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}  
  
public class Main10 {  
    static void withdraw(int balance, int amount) throws InsufficientBalanceException {  
        if (amount > balance) {  
            throw new InsufficientBalanceException("Insufficient balance in your account");  
        } else {  
            System.out.println("Withdrawal successful! Remaining balance: " + (balance - amount));  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            withdraw(5000, 7000); // change values to test  
        } catch (InsufficientBalanceException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        }  
    }  
}
```

Output :



```
D:\web_file>javac Main10.java  
D:\web_file>java Main10  
Exception caught: Insufficient balance in your account
```

Program No. 30

Objective: Write a program in Java to reading and writing in file using byte stream.

Code Implementation (Source Code):

```
import java.io.*;

public class ByteStreamDemo {

    public static void main(String[] args) {

        try {

            FileOutputStream fos = new FileOutputStream("sample.txt");

            String data = "Hello, this is byte stream example!";

            byte[] bytes = data.getBytes();

            fos.write(bytes);

            fos.close();

            System.out.println("Data written to file successfully.");

            FileInputStream fis = new FileInputStream("sample.txt");

            int i;

            System.out.println("Reading data from file:");

            while ((i = fis.read()) != -1) {

                System.out.print((char) i);

            }

            fis.close();

        } catch (IOException e) {

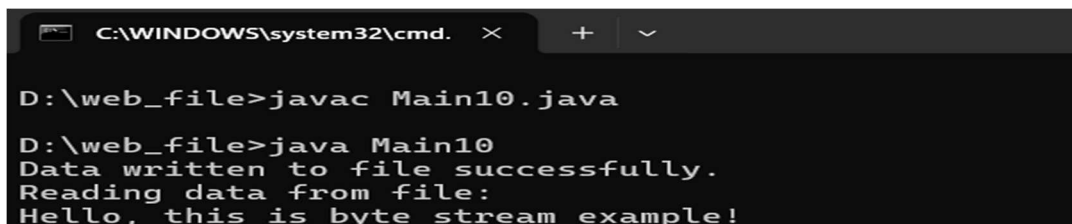
            System.out.println("Error: " + e.getMessage());

        }

    }

}
```

Output :



```
C:\WINDOWS\system32\cmd. x + v

D:\web_file>javac Main10.java

D:\web_file>java Main10
Data written to file successfully.
Reading data from file:
Hello, this is byte stream example!
```

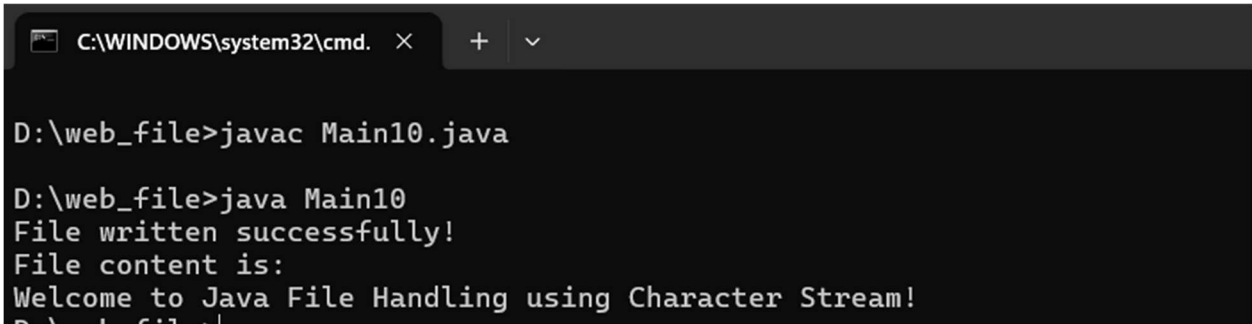
Program No. 31

Objective: Write a program in Java to reading and writing in file using character stream.

Code Implementation (Source Code):

```
import java.io.*;
public class CharacterStreamDemo {
public static void main(String[] args) {
try {
FileWriter fw = new FileWriter("sample.txt");
String data = "Welcome to Java File Handling using Character Stream!";
fw.write(data);
fw.close();
System.out.println("File written successfully!");
FileReader fr = new FileReader("sample.txt");
int i;
System.out.println("File content is:");
while ((i = fr.read()) != -1) {
System.out.print((char) i);
}
fr.close();
} catch (IOException e) {
System.out.println("Error occurred: " + e.getMessage());
}}}
```

Output :



```
C:\WINDOWS\system32\cmd.  ×  +  v

D:\web_file>javac Main10.java

D:\web_file>java Main10
File written successfully!
File content is:
Welcome to Java File Handling using Character Stream!
```

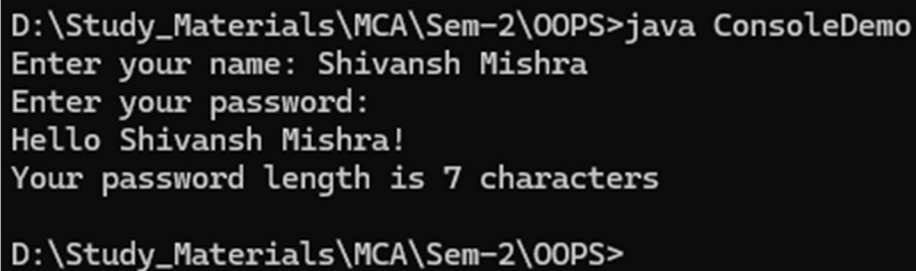
Program No. 32

Objective: Write a program in Java to reading and writing through console class.

Code Implementation (Source Code):

```
import java.io.*;
public class ConsoleDemo {
    public static void main(String[] args) {
        Console con = System.console();
        if (con == null) {
            System.out.println("Console not available");
            return;
        }
        String name = con.readLine("Enter your name: ");
        char[] password = con.readPassword("Enter your password: ");
        con.printf("Hello %s!\n", name);
        con.printf("Your password length is %d characters\n", password.length);
    }
}
```

Output :



```
D:\Study_Materials\MCA\Sem-2\OOPS>java ConsoleDemo
Enter your name: Shivansh Mishra
Enter your password:
Hello Shivansh Mishra!
Your password length is 7 characters

D:\Study_Materials\MCA\Sem-2\OOPS>
```

Program No. 33

Objective: Write a program in Java how to create thread using Thread Class.

Code Implementation (Source Code):

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread: " + i);  
            try { Thread.sleep(500); }  
            catch (InterruptedException e) { System.out.println(e); }  
        }  
    }  
}  
  
public class Q33 {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Main: " + i);  
        }  
    }  
}
```

Output:

```
D:\MCA\wt>javac Q33.java
```

```
D:\MCA\wt>java Q33
```

```
Main: 1
```

```
Thread: 1
```

```
Main: 2
```

```
Main: 3
```

```
Main: 4
```

```
Main: 5
```

```
Thread: 2
```

```
Thread: 3
```

```
Thread: 4
```

```
Thread: 5
```

Program No. 34

Objective: Write a program in Java how to create thread using Runnable interface.

Code Implementation (Source Code):

```
class MyRunnable implements Runnable {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Runnable Thread: " + i);  
            try { Thread.sleep(400); }  
            catch (InterruptedException e) { System.out.println(e); }  
        }  
    }  
}
```

```
public class Q34 {  
    public static void main(String[] args) {  
        MyRunnable r = new MyRunnable();  
        Thread t = new Thread(r);  
        t.start();  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Main Thread: " + i);  
        }  
    }  
}
```


Output:

```
D:\MCA\wt>javac Q34.java
```

```
D:\MCA\wt>java Q34
```

```
Runnable Thread: 1
```

```
Main Thread: 1
```

```
Main Thread: 2
```

```
Main Thread: 3
```

```
Main Thread: 4
```

```
Main Thread: 5
```

```
Runnable Thread: 2
```

```
Runnable Thread: 3
```

```
Runnable Thread: 4
```

```
Runnable Thread: 5
```

Program No. 35

Objective: Write a program in Java to implement the multithreading.

Code Implementation (Source Code):

```
class Thread1 extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++)  
            System.out.println("Thread 1 - Count: " + i);  
    }  
}
```

```
class Thread2 extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++)  
            System.out.println("Thread 2 - Count: " + i);  
    }  
}
```

```
class Thread3 extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++)  
            System.out.println("Thread 3 - Count: " + i);  
    }  
}
```

```
public class Q35 {
```

```
public static void main(String[] args) {  
    Thread1 t1 = new Thread1();  
    Thread2 t2 = new Thread2();  
    Thread3 t3 = new Thread3();  
    t1.start();  
    t2.start();  
    t3.start();  
}  
}
```

Output:

```
D:\MCA\wt>javac Q35.java
```

```
D:\MCA\wt>java Q35
```

```
Thread 3 - Count: 1
```

```
Thread 3 - Count: 2
```

```
Thread 2 - Count: 1
```

```
Thread 2 - Count: 2
```

```
Thread 2 - Count: 3
```

```
Thread 1 - Count: 1
```

```
Thread 3 - Count: 3
```

```
Thread 1 - Count: 2
```

```
Thread 1 - Count: 3
```

```
D:\MCA\wt>|
```

Program No. 36

Objective: Write a program in Java to achieve synchronization in threads.

Code Implementation (Source Code):

```
class Counter {  
    private int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
  
    int getCount() { return count; }  
}  
  
class MyThread extends Thread {  
    Counter c;  
    MyThread(Counter c) { this.c = c; }  
  
    public void run() {  
        for (int i = 0; i < 1000; i++)  
            c.increment();  
    }  
}  
  
public class Q36 {  
    public static void main(String[] args) throws InterruptedException {
```

```
Counter c = new Counter();  
MyThread t1 = new MyThread(c);  
MyThread t2 = new MyThread(c);  
t1.start(); t2.start();  
t1.join(); t2.join();  
System.out.println("Final Count: " + c.getCount());  
}  
}
```

Output:

```
D:\MCA\wt>javac Q36.java  
  
D:\MCA\wt>java Q36  
Final Count: 2000  
  
D:\MCA\wt>|
```

Program No. 37

Objective: Write a program in Java to implement the concept of Priorities in threads.

Code Implementation (Source Code):

```
class PriorityThread extends Thread {
    PriorityThread(String name) {
        super(name);
    }

    public void run() {
        System.out.println(getName() + " priority: " + getPriority() + " running.");
    }
}

public class Q37 {
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread("Low Priority");
        PriorityThread t2 = new PriorityThread("Normal Priority");
        PriorityThread t3 = new PriorityThread("High Priority");

        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.NORM_PRIORITY); // 5
        t3.setPriority(Thread.MAX_PRIORITY); // 10

        t1.start(); t2.start(); t3.start();
    }
}
```

}

Output:

```
D:\MCA\wt>javac Q37.java

D:\MCA\wt>java Q37
High Priority priority: 10 running.
Normal Priority priority: 5 running.
Low Priority priority: 1 running.

D:\MCA\wt>|
```

Program No. 38

Objective: Write a program in Java to illustrate the concept of Generic Programming.

Code Implementation (Source Code):

```
class GenericBox<T> {
    private T value;

    GenericBox(T value) { this.value = value; }

    T getValue() { return value; }

    void display() {
        System.out.println("Value: " + value + " | Type: " + value.getClass().getName());
    }
}

class GenericPair<K, V> {
    K key; V val;

    GenericPair(K key, V val) { this.key = key; this.val = val; }

    void display() { System.out.println("Key: " + key + ", Value: " + val); }
}

public class Q38 {
    static <T extends Comparable<T>> T findMax(T a, T b) {
        return (a.compareTo(b) > 0) ? a : b;
    }
}
```



```

public static void main(String[] args) {
    GenericBox<Integer> intBox = new GenericBox<>(42);
    GenericBox<String> strBox = new GenericBox<>("Java");
    intBox.display();
    strBox.display();

    GenericPair<String, Integer> pair = new GenericPair<>("Roll No", 31);
    pair.display();

    System.out.println("Max(10, 20): " + findMax(10, 20));
    System.out.println("Max(Apple, Mango): " + findMax("Apple", "Mango"));
}
}

```

Output:

```

D:\MCA\wt>javac Q38.java

D:\MCA\wt>java Q38
Value: 42 | Type: java.lang.Integer
Value: Java | Type: java.lang.String
Key: Roll No, Value: 31
Max(10, 20): 20
Max(Apple, Mango): Mango

D:\MCA\wt>|

```

Program No. 39

Objective: Write a program in Java to illustrate the concept of event handling (using various event handlers).

Code Implementation (Source Code):

```
import java.awt.*;
import java.awt.event.*;

public class Q39 extends Frame implements ActionListener, MouseListener, KeyListener {

    Label label;
    Button btn;
    TextField tf;

    Q39() {
        setLayout(new FlowLayout());
        label = new Label("Event Handling Demo");
        btn = new Button("Click Me");
        tf = new TextField(20);

        btn.addActionListener(this);
        addMouseListener(this);
        tf.addKeyListener(this);

        add(label); add(btn); add(tf);
        setSize(400, 200);
        setTitle("Event Handling");
    }
}
```

```

setVisible(true);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) { dispose(); }
});
}

public void actionPerformed(ActionEvent e) {
    label.setText("Button Clicked!");
}

public void mouseClicked(MouseEvent e) {
    label.setText("Mouse Clicked at (" + e.getX() + "," + e.getY() + ")");
}

public void mouseEntered(MouseEvent e) {}
public void mouseExited(MouseEvent e) {}
public void mousePressed(MouseEvent e) {}
public void mouseReleased(MouseEvent e) {}

public void keyTyped(KeyEvent e) {
    label.setText("Key Typed: " + e.getKeyChar());
}

public void keyPressed(KeyEvent e) {}
public void keyReleased(KeyEvent e) {}

public static void main(String[] args) {

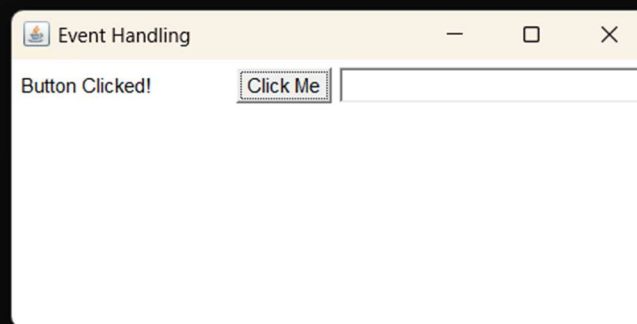
```

```
        new Q39();  
    }  
}
```

Output:

```
D:\MCA\wt>javac Q39.java
```

```
D:\MCA\wt>java Q39
```



Program No. 40

Objective: Create a simple student registration application using various swing components (like: JFrame, JButton, JLabel, text fields, text areas, check box and radio buttons).

Code Implementation (Source Code):

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Q40 extends JFrame implements ActionListener {

    JLabel nameLabel, rollLabel, genderLabel, courseLabel, addressLabel;
    JTextField nameField, rollField;
    JRadioButton male, female;
    ButtonGroup genderGroup;
    JCheckBox java, python, cpp;
    JTextArea addressArea;
    JButton submit, clear;
    JTextArea output;

    Q40() {
        setTitle("Student Registration Form");
        setSize(500, 600);
        setLayout(new GridLayout(0, 2, 5, 5));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        nameLabel = new JLabel("Name:");
```

```

nameField = new JTextField();
rollLabel = new JLabel("Roll No:");
rollField = new JTextField();
genderLabel = new JLabel("Gender:");
male = new JRadioButton("Male");
female = new JRadioButton("Female");
genderGroup = new ButtonGroup();
genderGroup.add(male); genderGroup.add(female);
JPanel genderPanel = new JPanel();
genderPanel.add(male); genderPanel.add(female);
courseLabel = new JLabel("Courses:");
java = new JCheckBox("Java");
python = new JCheckBox("Python");
cpp = new JCheckBox("C++");
JPanel coursePanel = new JPanel();
coursePanel.add(java); coursePanel.add(python); coursePanel.add(cpp);
addressLabel = new JLabel("Address:");
addressArea = new JTextArea(3, 20);
submit = new JButton("Submit");
clear = new JButton("Clear");
output = new JTextArea(5, 30);
output.setEditable(false);

submit.addActionListener(this);
clear.addActionListener(this);

add(nameLabel); add(nameField);
add(rollLabel); add(rollField);

```

```

add(genderLabel); add(genderPanel);

add(courseLabel); add(coursePanel);

add(addressLabel); add(new JScrollPane(addressArea));

add(submit); add(clear);

add(new JLabel("Output:")); add(new JScrollPane(output));

setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submit) {
        String gender = male.isSelected() ? "Male" : female.isSelected() ? "Female" : "Not
selected";

        String courses = (java.isSelected() ? "Java " : "") +
            (python.isSelected() ? "Python " : "") +
            (cpp.isSelected() ? "C++ " : "");

        output.setText("Name: " + nameField.getText() + "\n" +
            "Roll No: " + rollField.getText() + "\n" +
            "Gender: " + gender + "\n" +
            "Courses: " + courses + "\n" +
            "Address: " + addressArea.getText());
    } else {
        nameField.setText(""); rollField.setText("");
        genderGroup.clearSelection();
        java.setSelected(false); python.setSelected(false); cpp.setSelected(false);
        addressArea.setText(""); output.setText("");
    }
}
}

```

```

public static void main(String[] args) {
    new Q40();
}
}

```

Output:

Student Registration Form

Name: Anshika Trivedi

Roll No: 31

Gender: ☐ Male ☒ Female

Courses: ☒ Java ☐ Python ☒ C++

Address: Fauzi nagar colony Lucknow

Submit **Clear**

Output:
 Name: Anshika Trivedi
 Roll No: 31
 Gender: Female
 Courses: Java C++
 Address: Fauzi nagar colony Lucknow